Lab Work 1
Secure REST API, CI/CD and a Real CVE

FastAPI + JWT · GitHub Actions · CVE-2021-41773 reproduction and fix

Name	Zhang San
Student ID	12345678
Variant	1
Repository	github.com/huangweijie429/Information-Security-Lab1

1. Purpose

This work covers the full security lifecycle of an API: designing protection against the OWASP Top 10, wiring security tools into CI/CD, reproducing a real CVE in a controlled environment (the same configuration Vulhub uses), analysing its root cause, applying a fix and verifying both that the attack fails and that the application still works.

2. API design and implementation

Stack: **FastAPI + SQLite (stdlib `sqlite3`) + bcrypt + PyJWT**, code in `work1-api/`.

Method	Path	Auth	Description
GET	`/health`	no	liveness probe
POST	`/auth/login`	no	accepts `{username, password}`, returns a JWT
GET	`/api/data`	Bearer	posts list, optional `?q=` filter
GET	`/api/me`	Bearer	current user profile (the self-chosen third endpoint)

3. Security measures

Risk	Implementation
SQL injection	Every statement uses `?` placeholders (login query and the `q` filter); no string concatenation. The username field is additionally allow-listed with `^[A-Za-z0-9_.-]+$`.
XSS	All user-controlled fields are escaped with `html.escape()` before leaving the API; a regression test asserts that `<b>` becomes `<b>`.
Broken authentication	bcrypt (cost 12) password hashes; HS256 JWT with `exp` / `iss`; every protected route is guarded by an `HTTPBearer` dependency that returns 401 for missing or forged tokens.

4. CI/CD pipeline

`.github/workflows/ci.yml` runs on every push and pull request:

1. **SAST**: `bandit -r app` — no issues identified
2. **SCA**: `pip-audit -r requirements.txt --strict` — no known vulnerabilities
3. **Tests**: `pytest -q` — 10 tests, all passing (SQLi / XSS / auth regressions)

Run records are on the repository's Actions page (first fully green run: 39257053599).

5. CVE-2021-41773 reproduction

Target: path traversal in Apache HTTP Server 2.4.49, which also enables CGI command execution with the right configuration. The environment (`work1-cve/Dockerfile`) mirrors the Vulhub setup: `mod_cgi` and `ScriptAlias` enabled, and `<Directory />` set to `Require all granted`.

Request	Vulnerable 2.4.49	Fixed 2.4.51
`GET /cgi-bin/.%2e/.%2e/.%2e/.%2e/etc/passwd`	**HTTP 200**, returns `root:x:0:0:root:/root:/bin/bash`	HTTP 400, no file content
`POST …/bin/sh` with body `echo Content-Type: text/plain; echo; id`	**HTTP 200**, `uid=1(daemon) gid=1(daemon)`	HTTP 400, no output
`GET /cgi-bin/hello.sh` (regression)	200 hello from cgi	200 hello from cgi (functionality intact)

Full commands and outputs: `work1-cve/evidence/01-05-*.txt`.

6. Root cause

Apache 2.4.49 rewrote path normalisation (`ap_normalize_path`). The new code only expanded `%2e` segments for paths that start inside the document root and did not reject `..` sequences that escape it. A request such as `/.%2e/.%2e/…` was therefore normalised into `../../..` after the authorisation check but before the file lookup, bypassing access control. Two configuration choices turn file disclosure into code execution:

1. `<Directory />` grants access to the whole filesystem, so the escaped path passes authorisation;
2. `mod_cgi` and `ScriptAlias` are enabled, so a traversed executable is run with the request body as input.

The defect class is therefore **improper input handling** (incomplete canonicalisation of a user-controlled path), not a memory-safety bug.

7. Fix and verification

Layer	Change	Why it works
Patch	image upgraded `httpd:2.4.49 → 2.4.51`	2.4.51 rejects any normalised path that escapes the document root and also blocks the double-encoded `%252e` bypass of 2.4.50
Hardening	`<Directory />` changed to `Require all denied`; CGI restricted to cgi-bin	removes the second precondition: even if a future normalisation bug appears, a traversed path cannot be authorised

After the fix the same PoC returns 400, while the home page and legitimate CGI still work (regression passed). `poc.sh` verifies any instance in one command: exit code 1 for the vulnerable build, 0 for the fixed one.

8. Conclusion

Parameterized queries, output escaping and strong authentication push application-level injection and script execution risks very low, but they cannot stop **defects in the infrastructure itself**. This CVE teaches two lessons at once: patch promptly (remove the known defect), and apply least privilege (even without the patch, the impact drops from RCE to a 403). Together they are what makes a regression test able to prove both "the attack fails" and "the feature still works".